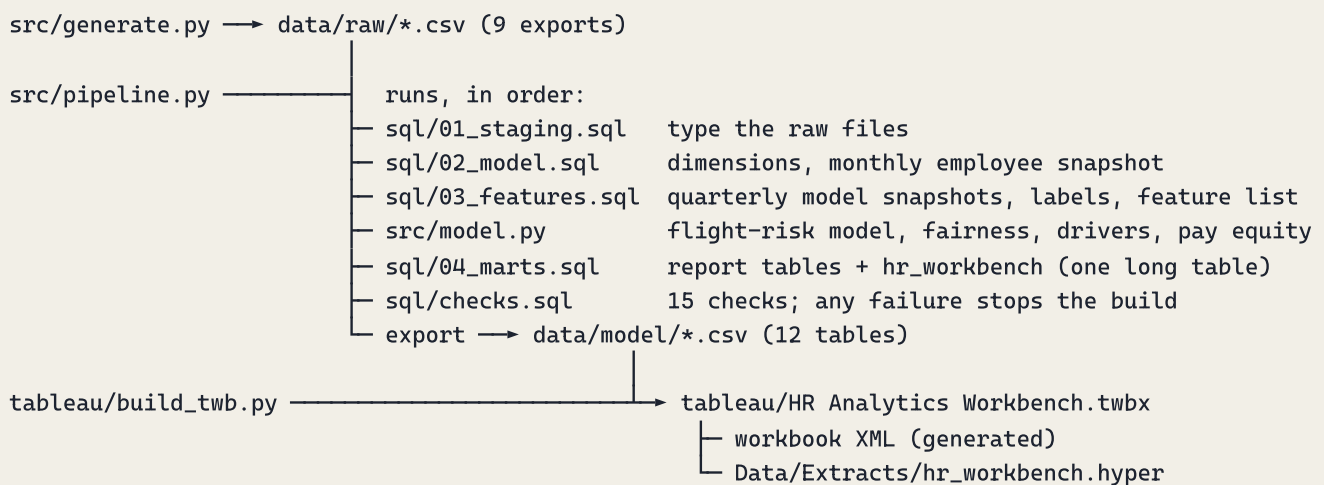


How it works

A walkthrough of the project for reviewers: what each part does, how data moves from raw HR exports to the Tableau workbench, how correctness is enforced, and where your eyes are most useful.

Companion to the case study. Paths are relative to the project folder.

1. The flow in one picture



Everything downstream of `generate.py` is deterministic: same seed, same data, same model, same numbers. Three consecutive pipeline runs produce byte-identical exports, and the workbook XML is byte-identical across builds. The Hyper extract file differs in its internal metadata between builds, but holds identical rows.

2. Running it

```

python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python src\generate.py          # writes data/raw
.venv\Scripts\python src\pipeline.py          # builds hr.duckdb, trains, checks, exports
.venv\Scripts\python tableau\build_twb.py     # writes tableau/HR Analytics Workbench.twbx
  
```

Open `tableau\HR Analytics Workbench.twbx` in Tableau Public 2026.2 (free). The data travels inside the file, so no paths need editing. Tested on Tableau Public 2026.2.2 for Windows.

Expected `pipeline.py` output: three "built" lines, "trained model: Gradient boosting, test ROC AUC 0.658, top-decile capture 20.4%", "built 04_marts.sql", 15 lines starting with `pass`, then "exported 12 tables".

3. Stage by stage

3.1 Generator: `src/generate.py`

Simulates a 2,150-person firm from April 2022 to March 2026, month by month, and writes what an HR analyst could pull from an HRIS, an ATS, a timesheet system and a survey tool.

MECHANISM	HOW IT WORKS
Reproducibility	One seeded random generator (<code>SEED = 11</code>). Every person, date, salary and response comes from it.
Opening population	2,150 people with a realistic level mix (30% L1 down to 2% L6), seven departments, 60/40 Hyderabad/Bengaluru, and two years of back history so growth features exist from the first month.
Pay	Market band midpoints by level, department, location and year, moving 8% a year. Starting pay is the band midpoint with noise; women's is multiplied by 0.965 (the planted gap). Each April, raises scale with rating and the year's market rate; promotions add 15 to 22%.
Promotions	Probability by rating (0% for a 1 to 45% for a 5), only after 18 months in level; women's probability is multiplied by 0.82 (planted).
Resignations	A monthly hazard: the year's base rate (highest in FY 2022-23) multiplied up for pay below band, pay growth under 12% in two years, a long wait for promotion, commutes over 75 minutes, the 1 to 3 year tenure window, low recent utilisation and strong performers paid below band. A hidden satisfaction factor adds noise the model can't see.
Dismissals	Mostly low ratings, plus a bench restructuring in Delivery in late 2023.
Hiring	85% of exits open a backfill requisition; about four growth requisitions a month; a January campus drive. Time to hire depends on level, is slower in Data and Analytics and in FY 2022-23. Requisitions still open at the end have no hire date.
eNPS	Quarterly survey, 55% response, score driven by the hidden satisfaction factor and a year effect. Responses carry department, location and level band only.

3.2 Staging: `sql/01_staging.sql`

Reads each CSV with explicit types. Ratings get a `known_from` date (the April after the review cycle) so later stages can't use a rating before it existed. No business logic lives here.

3.3 Model: `sql/02_model.sql`

TABLE	BUILT HOW
<code>dim_month</code>	48 months with month end and the Indian financial year label (April to March).
<code>dim_employee</code>	Every person ever employed, with their exit (if any).
<code>fact_employee_month</code>	One row per person per month end on the books (107,084 rows). Department, level, last promotion, pay, pay 24 months earlier, latest rating and the one before are attached with <code>ASOF</code> joins: each takes the latest record dated on or before the month end. Utilisation over the last 3 months and bench months over the last 12 come from correlated subqueries. Compa-ratio is pay divided by the band midpoint for that level, department, location and year.
<code>fact_requisition</code>	Requisitions with time to hire (opened to offer accepted) and time to fill (opened to join).

3.4 Features and the model: `sql/03_features.sql` , `src/model.py`

STEP	HOW IT WORKS
Snapshots	Every employee at every quarter end. Label: resigned within the next six months. A dismissal inside the window marks the row censored (outcome unknowable) and it is dropped. Rows whose six-month window runs past March 2026 aren't labelled.
Features	14, listed in the <code>model_features</code> table: 10 numeric, 4 categorical. <code>model.py</code> asserts none is protected, and a check repeats the test.
Split	Out of time: train on snapshots to September 2023 (11,946 rows), test from March 2024 (16,233 rows). The gap means no training label overlaps the test period.
Models	Logistic regression (median imputation with missing flags, scaling, one-hot) and histogram gradient boosting (depth 3, 300 iterations). Logistic is preferred unless the other beats it by at least 0.01 AUC; gradient boosting won by 0.014.
Outputs	Metrics, calibration by decile, permutation importance, fairness by gender, then a refit on all labelled history to score everyone on the books at 31 March 2026. Top 10% of scores = High, next 20% = Medium.
Drivers	For each person, the model is re-scored with one lever reset to a typical value (median for pay, pay growth, promotion wait, commute and utilisation; zero for bench time and rating change; "3 to 6 years" for tenure stage). Levers that cut the score by more than 0.5 percentage points are listed, biggest first, up to three.
Pay equity	Ordinary least squares on log pay with level, department and location dummies, rating and tenure, plus a "woman" flag; the flag's coefficient becomes the adjusted ratio with a 95% interval from classical standard errors.

All queries feeding the model sort their rows. Without that, DuckDB's arbitrary row order changed the gradient-boosting validation split and the results moved slightly on every run.

3.5 Marts: `sql/04_marts.sql`

MART	DEFINITION
<code>mart_exits</code> , <code>mart_hires</code>	Each movement with the job held at the time (ASOF on job history). Regretted = a resignation by someone last rated 4 or 5.
<code>mart_headcount_monthly</code>	A dense grid: every department × level × location × gender segment in every month, with headcount, hires, exits by type, and rolling 12-month exits and headcount (NULL before March 2023). Attrition = exits / headcount × 12, summed over whatever is filtered.
<code>mart_current_employees</code>	Everyone on the books at 31 March 2026 with pay position (below, within or above band at ±10% of midpoint) and risk score, band and drivers.
<code>mart_requisitions</code> , <code>mart_enps</code>	Requisitions by financial year; survey responses with Promoter (9 to 10), Passive (7 to 8) and Detractor (0 to 6).
<code>hr_workbench</code>	One long table for Tableau: record types Month, Employee, Requisition, Survey and Driver (one row per lever per top-10% employee), sharing department, location, level band and financial year columns, so one set of filters reaches every view.

3.6 Checks: `sql/checks.sql`

Each check is a query that returns the rows breaking a rule. `pipeline.py` runs them all, prints pass or fail, and exits with an error before exporting anything if any fail.

CHECK	LOGIC
Headcount rolls forward	For every month: previous headcount + hires – exits = this headcount.
Snapshot completeness	Every employee-month has a level, a salary and a market band.
Exits and hires are sane	Nobody leaves before joining or twice; exactly one hire record each; promotions move exactly one level.
ATS matches HRIS	Each filled requisition's hire has the same date, department, level and location.
Ranges	Ratings 1 to 5, eNPS scores 0 to 10.
Survey anonymity	No column resembling an employee ID exists in the survey table.
Mart totals	Exits and hires in the monthly mart equal the source counts in the window.
Rolling values	Rolling 12-month exits and headcount equal a direct count from source, for every department and month, and are NULL before 12 months of history.
Model hygiene	No protected feature; test starts after every training label resolved; one score in [0, 1] per current employee; pay equity covers every current woman and man.
Workbench reconciles	Row counts per record type equal their source marts.

The checks were themselves tested by breaking the data each one guards (for example, a rolling value off by one, a value before 12 months of history, a deleted workbench row). Each was caught.

3.7 Tableau generation: `tableau/build_twb.py` , `tableau/make_extract.py`

- **Format.** The workbook is written in Tableau's current XML (version 18.1), matched element by element to a copy Tableau Public saved itself. Older-format files load through Tableau's upgrade path, but that path drops custom colour palettes.
- **Data.** `make_extract.py` writes `hr_workbench` to a Hyper extract with Tableau's Hyper API (telemetry off). Tableau Public only accepts extracts. Column types come from the DuckDB schema, so the workbook can't drift from the data.
- **Filters.** Department, location, level and year are list parameters. One calculated field, *In scope*, combines them, and every sheet filters on it plus its record type. Year doesn't apply to Employee and Driver rows, which describe today's workforce.
- **Calculations.** 15 calculated fields, among them attrition (annualised and rolling 12 months), voluntary share, women's share, median time to hire, eNPS and the raw pay ratio. Rates resting on fewer than five records return NULL and are filtered out (`MIN_N`).
- **Look.** Colours per category live in the data source's palette; titles, axis formats, column widths and the dashboard grid are all set in code.
- **Test switch.** `--preset Department="Data and Analytics"` builds a copy that opens with filters already set; `tableau/filter_check.py` prints the numbers it should show, computed independently in DuckDB.

3.8 Helper scripts

SCRIPT	PURPOSE
<code>tableau/validate_twb.py</code>	Validates the workbook XML against Tableau's published schema (in <code>tableau/schema</code>). A small set of errors is expected: Tableau's own saved file produces the identical set, because the schema relies on feature switches the validator can't evaluate.
<code>tableau/open_twb.ps1</code>	Opens a workbook in a new Tableau Public window, captures that window, reads its log for errors, and closes only that window.
<code>tableau/capture_docs.ps1</code>	Case-study images: native-size captures cropped to the 1440 × 900 canvas, retaken if Tableau's editor outlines appear.
<code>docs/build_pdf.ps1</code>	Prints the case study to PDF with headless Edge and renders every page to an image for layout review.

4. Design choices worth challenging

CHOICE	WHY	TRADE-OFF
Synthetic data	Publishable and reproducible; planted effects let the methods be checked against a known answer.	The model can only find what was planted.
Six-month resignation label	Long enough to act on, short enough to be fresh.	Throws away the last six months of history for training and testing.
Dismissals censored, not negative	Someone dismissed might have resigned later; calling them a stayer is wrong.	Fewer labelled rows; a survival model would use them properly.
Risk shown as rank	The model over-predicts after the 2022 wave (calibration drift).	Loses the probability reading until the model is recalibrated.
Drivers by lever reset	Works for any model; only reports things HR can change.	Ignores interactions between levers; not a causal estimate.
Rolling rates in SQL	Exact under any filter, including year.	A denser monthly table (8,112 segment-months instead of only the non-empty ones).
One long table	Every filter reaches every view, no data blending.	Sparse columns; every calculation must test record type.
Generated workbook	Reproducible, reviewable in a diff.	Tableau's format is undocumented; the generator follows Tableau's own save and is tested on one Tableau version.

5. What to review

1. **Correctness.** Do the definitions in section 3.5 match how you'd define attrition, regretted exits and pay position?
2. **Leakage.** Can any feature in `fact_employee_month` see the future? Look at the `ASOF` joins and the rating `known_from` dates.

3. **Model.** Is the out-of-time split, the censoring and the model choice sound? Would you report something other than ROC AUC and top-decile capture?
4. **Fairness.** Is the reading of the fairness table (women flagged more because they are paid less) convincing? What else would you test?
5. **Checks.** Is there a failure mode the 15 checks would miss?
6. **Workbench.** Can an HR business partner answer "how is my department doing" within a few seconds? Is anything misleading?

6. Known limits

- The firm, its people and the market are synthetic; tax and benefits aren't modelled.
- Time to hire counts only filled requisitions, so the latest year looks faster than it is.
- The lever chart shows counts; a share would need a Tableau table calculation.
- The workbook is tested on Tableau Public 2026.2.2 for Windows only.